Chapter 7: Software Quality

Assurance & Testing

Ensuring Reliability, Correctness,

and Performance

Focus: From "It works on my

machine" to "It works for the user."

Learning Objectives

• Overview: What is testing and why do we do it?

• Concepts: Failures, Faults, and Errors.

• Activities: The levels of testing (Unit to Acceptance).

• Managing Testing: Plans and Roles.

• OO Impact: How classes and objects change testing

strategies.

• Types: Black-box vs. White-box testing.

7.1 An Overview of Testing

• Verification vs. Validation

• Software Quality Assurance (SQA): The systematic process

of checking if software meets specified requirements.

• Verification: "Are we building the product right?" (Checking

code against design).

• Validation: "Are we building the right product?" (Checking

software against user needs).

• Example:

• Verification: A developer reviews the code for the

calculateGPA() function to ensure it follows the math

formula correctly.

• Validation: We show the "Grade Report" screen to the Registrar to ask, "Is this the layout and information you need to print transcripts?"

7.2 Testing Concepts: The Chain of Defects

Reliability Terminology It is crucial to distinguish between the cause and the effect of a bug.

• Fault (Bug): The mechanical/algorithmic defect in the code.

• Error: The incorrect internal state of the system caused by the fault.

• Failure: The external, visible deviation from expected behavior.

Example:

• Fault: The programmer types if (capacity < 0) instead of if (capacity == 0) in the logic.

• Error: The variable isFull remains false even when the course has 0 seats left.

• Failure: The system allows a 31st student to register for a 30-seat class.

7.2 Testing Concepts: Test Components

Tools of the Trade

• Test Case: A set of inputs, execution preconditions, and expected outcomes.

• Test Stub: A dummy piece of code that simulates a component that is called by the code being tested.

• Test Driver: A dummy piece of code that calls the code being tested.

Example:

• We are testing the CourseManager class, but the Database isn't built yet.

• Stub: We write a FakeDatabase that just returns "Success" whenever CourseManager tries to save data.

• Driver: We write a simple script MainTest.java that manually calls CourseManager.addStudent("John") to see if it works.

7.3 Testing Activities (The V-Model)

Levels of Testing

• Unit Testing: Testing individual components (classes/methods) in isolation.

• Integration Testing: Testing how those components interact (interfaces).

• System Testing: Testing the entire system as a whole against requirements.

• Acceptance Testing: The client tests the system to decide if they will pay for it.

Example:

• Unit: Testing the Student.getName() method.

• Integration: Testing if Student object can successfully be saved into the Course list.

• System:Testing the full "Register for Semester" workflow (Login ->

Search -> Add -> Print).

• Acceptance: The University Admin uses the system for a week during a pilot phase.

7.4 Managing Testing

Organization and Documentation

• Test Plan: A document describing scope, approach, resources, and schedule.

• The Tester Role: Ideally, developers should not be the only ones testing their own code (blind spots).

• Bug Tracking: Using tools (like Jira) to record defects, assign priority, and track fixes.

Example:

• Goal: "Ensure the system can handle 5,000 simultaneous students."

• Plan: "We will run load tests on Friday night using a server simulator."

• Role: The "QA Engineer" tries to break the login screen by entering emojis and huge text, something the developer didn't think to try.

7.5 Impact of Object Oriented Testing

Why is OO Testing Harder?

• Encapsulation: Private data is hidden. How do we check if a private counter is correct without a public getter?

• Inheritance: If we test a User class, we must also re-test Student and Professor subclasses because they might

override methods differently.

• Polymorphism: A method call person.register() could execute different code depending on whether person is a Student or Staff.

Example:

• We tested User.login() and it works.

• However, Student inherits from User. If Student overrides login() to check for "Tuition Payment status," the original test for User is insufficient. We must write a specific test for Student as well.

7.6 Types of Testing (Strategy)

Black Box vs. White Box

• Black Box (Functional): Testing without looking at the code. Focuses on Input/Output.

• White Box (Structural): Testing with full knowledge of the code logic (branches, loops).

Example:

• Black Box: The tester sees a "Login" box. They try entering "User1" and "Password123" to see if it logs in. They don't care how it works.

• White Box: The developer looks at the code: if (password.length < 8). They specifically write a test case with a 7-character password to ensure that specific if statement triggers correctly.

7.6 Types of Testing (Specialized)

Regression and Performance

• Regression Testing: Re-running old tests after a code change to ensure nothing broke ("Did we create a new bug while fixing an old one?").

• Performance/Stress Testing: Testing limits (speed, load).

Example:

• Regression: We fixed a bug in the "Print Transcript" feature. We must re-test the "Enter Grades" feature because they share the same database tables, just to be safe.

• Stress Test: We simulate 10,000 students hitting the "Register" button at the exact same second to see if the server crashes.

Case Study: The "FlexiEnroll" Registration System

• Context: You are the Lead SQA Engineer for "FlexiEnroll," a simplified university registration platform. The university has mandated that the system must be strictly automated—no manual overrides are allowed. If the system rules say "No," the registration is rejected.

• The System Rules (The "Spec"):

• Authentication: Students must log in with a 6-digit Student ID and a password.

• Course Search: Students can view courses. Each

course displays:– Course ID(e.g., CS101)– Capacity (e.g., 30 seats)– Status (Open, Full, or Waitlisted)

• Registration Logic: To successfully add a course, the

system checks:– Availability: Is there a seat? If not, the student is prompted to join the

Waitlist (max 10 spots).– Prerequisites: Does the student have a passing grade (C or better) in

the required prerequisite course?– Time Conflicts: Does the course overlap with any currently scheduled

course?– Max Credits: A student cannot exceed 18 credits per semester.

• Dropping/Withdrawing: Students can "Drop" with

no penalty in the first 7 days.– After 7 days, they can only "Withdraw," which results in a "W" grade.

Directions for Students: SQA Lab Activity

• Objective: Apply Software Quality Assurance

techniques to verify the "FlexiEnroll" system. Provide

specific, professional answers for each section.

• Part 1: Boundary Value Analysis (BVA)

• Goal: Test the limits where code usually breaks.

• The Credit Limit: The max limit is 18 credits. Create a

test suite of four specific values (number of credits)

that you would test to ensure this logic works. Explain

why you chose these four.

• The Waitlist: The waitlist holds a maximum of 10

students. If the course is full and the waitlist has 10

people, what should happen to the 11th person? Write

a test case for this.

Part 2: Integration & Path Testing

• Goal: Test how different parts of the system talk

to each other.

• The "Prerequisite" Flow:  A student wants to

register for Advanced Java. The system must

check the Student Records Database for Intro to

Java. Explain how you would test this Integration.

What happens if the Student Records Database is

offline during the check?

• State Transition: A course status can change from

Open → Full → Waitlisted. Describe a sequence

of student actions that would move a course

from Full back to Open.

Part 3: White Box vs. Black Box

• Goal: Understand the difference between testing

"User Logic" and "Code Logic."

• Black Box: You are a student using the website.

Write a test case to check the Time Conflict rule.

What are your inputs? What is your expected

output on the screen?

• White Box: Look at this snippet of "pseudo-code"

used for the Capacity Check: if (enrolledStudents

>= capacity) { showWaitlistButton(); }– If the capacity is 30, and enrolledStudents is exactly

30, does the button show?– What happens if a bug caused enrolledStudents to be

31? Does the code still handle it correctly?

Part 4: Regression & Stress

• Goal: Test the system under pressure and after changes.

• Regression: The developers just fixed a bug in the "Login"

screen. Why is it important to re-test the "Course

Registration" page even though no code was changed

there?

• Stress: The enrollment window opens at 8:00 AM. 5,000

students will click "Submit" at the same time. Describe one

specific failure that might happen during this stress test

that wouldn't happen during a normal test.

Part 5: Identifying Ambiguity

• The "Drop" Rule: The spec says students can drop in the

"first 7 days."– As a tester, why is this rule ambiguous? (Hint: Does the 7 days

start from the day they registered, or the first day of the

semester? Does it end at 12:00 AM or 11:59 PM?)– How would you rewrite this rule to make it "testable"?

Possible solution for “FlexiEnroll”

Part 1: Boundary Value Analysis (BVA)

1. The Credit Limit (Max 18): The ideal test suite

uses the n-1, n, and n+1 approach.– 17 Credits: Should pass (just below the limit).– 18 Credits: Should pass (exactly at the limit).– 19 Credits: Should fail (first value to break the rule).– 0 Credits: Should pass or show a specific message

(minimum possible).– Bonus:-1 Credits (to check for input errors).

2. The Waitlist (M